

Atividades de Orientação a Objetos

PROVA A1 (3 PONTOS)

Observações

- Trabalho individual, **plágio/cola** será zerado **ambas as notas**;
- Códigos fontes deverão ser **upados no GIT** e passado link para professor;
 - Acesse e preencha o formulário:
<https://forms.gle/3HnxguWcFHweuwL69>
- Atenção ao prazo FINAL de entrega!

07/11/2025 - 19:00

ps.: Qualquer fonte que for upado no git após essa data, ou um GIT que não for preenchido no formulário até essa data/hora será desconsiderado e terá sua nota zerada.

Atividade 1:

Modelagem de um Sistema de Biblioteca

Escopo de estudo: **Classe, Objeto, Métodos e Atributos;**

Objetivo: Projetar e implementar classes básicas para um sistema de gerenciamento de biblioteca.

Descrição:

1. **Crie uma classe `Livro`** com pelo menos os seguintes atributos: `titulo` (String), `autor` (String), `editora`(String), `anoPublicacao` (int) e `disponivel` (boolean).
2. **Crie os métodos `emprestar()` e `devolver()`** para a classe `Livro`. Certifique-se das lógicas ideais para esses métodos.
3. **Crie uma classe `Membro`** com pelo menos os atributos: `nome` (String), `identificacao` (String) e `livrosEmprestados` (uma lista de objetos `Livro`).
4. **Crie métodos `pegarEmprestado(Livro livro)` e `devolverLivro(Livro livro)`** na classe `Membro`. Esses métodos devem interagir com os métodos da classe `Livro` e gerenciar a lista `livrosEmprestados` do membro.
5. **Instancie:** Crie 4 objetos `Livro` e 3 objetos `Membro`.
6. **Simule:** Realize múltiplas operações (ao menos 10, de empréstimo e devolução) entre os membros e os livros, demonstrando o funcionamento dos métodos e a alteração dos estados. Gere logs, do fluxo dessas operações no terminal para demonstrar sucessos e travas conforme lógicas adotadas.
7. **Demonstração do conhecimento:** Você deve explicar o que acontece na classe e porque acontece com suas palavras através de comentários em seus códigos fontes **COM SUAS PALAVRAS!**

Atividade 2: Sistema de Funcionários

Escopo de estudo: **Herança e Polimorfismo;**

Objetivo: Utilizar herança para modelar diferentes tipos de funcionários e demonstrar polimorfismo.

Descrição:

1. **Crie uma classe base abstrata `Funcionario`** com pelo menos os atributos: `nome` (String), `salario` (double) e `identificacao` (String).
2. **Declare um método abstrato `calcularSalario()`** nesta classe.
3. **Crie classes derivadas: `Gerente`, `Desenvolvedor` e `Estagiario`.**
4. **Implemente o método `calcularSalario()`** em cada classe derivada de forma diferente:
 - **`Gerente`:** Salário base + 20% de bônus.
 - **`Desenvolvedor`:** Salário base + 10% de bônus por projetos entregues (adicione um atributo `projetosEntregues` na classe `Desenvolvedor`).
 - **`Estagiario`:** Salário fixo.
5. **Instancie:** Crie uma lista de objetos `Funcionario` (do tipo `Funcionario`, mas instanciando `Gerente`, `Desenvolvedor`, `Estagiario`) pelo menos 4 de cada classe.
6. **Simule:** Percorra a lista de `Funcionario` e gere logs, que demonstre o polimorfismo adotado.
7. **Demonstração do conhecimento:** Você deve explicar o que acontece na classe e porque acontece com suas palavras através de comentários em seus códigos fontes **COM SUAS PALAVRAS!**

Atividade 3: Sistema de Pagamento

Escopo de estudo: **Abstração, Encapsulamento e Relacionamento;**

Objetivo: Projetar um sistema de pagamento utilizando abstração para métodos de pagamento e encapsulamento para proteger dados e métodos sensíveis.

Descrição:

1. **Crie uma classe `ContaBancaria`** com algumas propriedades e pelo menos o saldo da conta, seu histórico de movimentações.
2. **Crie uma interface `MeioPagamento`** com um método
 - **Crie classes concretas que implementem `MeioPagamento`** `CartaoCredito`, `CartaoDebito`, `BoletoBancario` e `Pix`.
 - Implemente o método **`processarPagamento`** em cada uma das 4 classes fazendo suas validações exclusivas.
3. **Aplique Abstração:** Certifique-se de ter todos atributos necessários para essa atividade.
 - Quais atributos devem ser úteis na `ContaBancaria` ou no `MeioPagamento` para um Cartão de Crédito ou Débito, Boleto e Pix serem executados?
4. **Aplique Encapsulamento:** Certifique-se de que todos os atributos e métodos sensíveis sejam privados e atributos acessíveis apenas por meio de métodos *getter* e *setter* (se necessário).
5. **Instancie:** Crie quantas instâncias forem necessárias para que movimente um total de 4 `ContaBancaria`
6. **Simule:** Mostre a flexibilidade do sistema, demonstre as validações dos meios de pagamento que foram construídos e exiba ao final o histórico de pagamentos.
7. **Demonstração do conhecimento:** Você deve explicar o que acontece na classe e porque acontece com suas palavras através de comentários em seus códigos fontes **COM SUAS PALAVRAS!**

Atividade 4: Refatoração de Código

Objetivo: Refatorar um trecho de código mal projetado para fazer o uso de OOP em todos os seus conceitos visto até o momento.

Descrição:

1. **Código Inicial (Problema):**

○ **SERÁ ENVIADO ARQUIVO.TS**

2. **Compreendendo a orientação a objetos**

Recrie o código com as boas práticas compreendidas em sala de aula.

3. **Descreva** quais foram as oportunidades de refatoração encontradas.

4. **Explique** o que você acredita que poderia acontecer a longo prazo com o código atual dessa atividade caso ele não se adequasse há um código com OOP.